

# AIMLCZG523- MLOps

## Assignment 1

### Heart Disease Prediction using an End-to-End MLOps Pipeline

---

#### Report

#### GitHub Url:

<https://github.com/2024ad05057/heart-disease-prediction-mlops.git>

#### Google Drive Recorded Video path:

<https://drive.google.com/file/d/1iRiSkODYIW6h09m7kZSLUndeXxW62eCw/view?usp=sharing>

Submitted By: R V Mahendran

Course: MLOps (AIMLCZG523)

Student ID: 2024AD05057

Mail ID: 2024AD05057@wilp.bits-pilani.ac.in

## Abstract

As part of this project implements an end-to-end MLOps pipeline for predicting heart disease using the UCI Heart Disease dataset. This workflow includes data preprocessing, model training, hyperparameter tuning, experiment tracking using MLflow, API development with FastAPI, containerization using Docker, deployment on Kubernetes (Minikube), monitoring with Prometheus and Grafana, logging, automated testing using Pytest, and continuous integration using GitHub Actions.

## 1. Introduction

Machine learning assist in predicting heart disease for healthcare professionals from clinical data. This project demonstrates model development, deployment and monitoring using MLOps practices.

## 2. Problem Statement

Develop a production-ready machine learning application capable of predicting heart disease and demonstrate an end-to-end MLOps workflow.

## 3. Objectives

- Preprocess the dataset
- Train and evaluate machine learning models
- Perform hyperparameter tuning
- Track experiments with MLflow
- Deploy using FastAPI
- Containerize using Docker

- Deploy on Kubernetes (Minikube)
- Implement logging
- Monitor using Prometheus and Grafana
- Automate testing with GitHub Actions

## 4. Project Architecture

Workflow:

Dataset → Data Preprocessing → Model Training →  
Hyperparameter Tuning → MLflow → FastAPI → Docker →  
Kubernetes → Logging → Prometheus → Grafana → GitHub  
Actions.

UCI Heart Disease Dataset



Data Preprocessing



Model Training

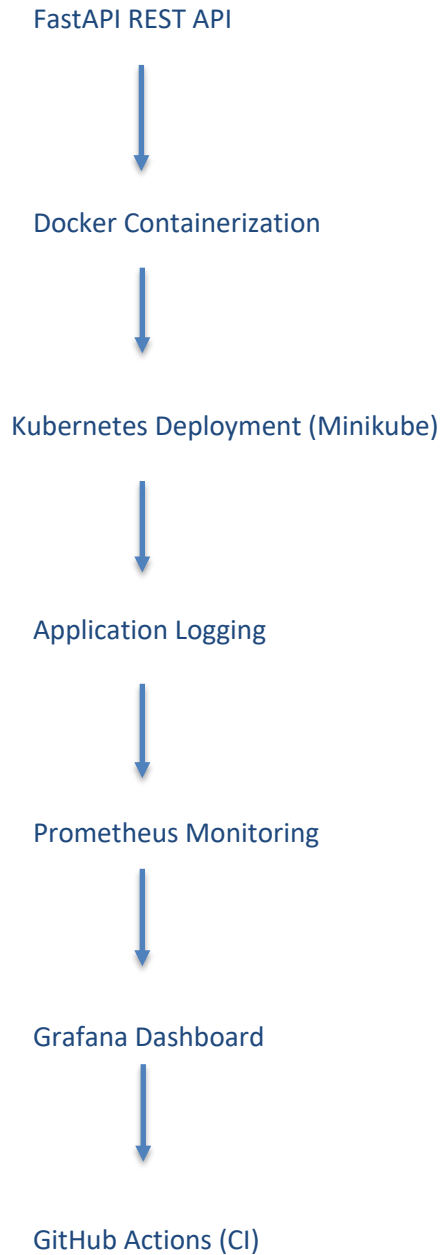


Hyperparameter Tuning



MLflow Experiment Tracking





## 5. Data Acquisition & Exploratory Data Analysis

### 5.1 Objective

The main objective here is download the dataset, understand it, perform data cleaning, and analyze the data distribution before training the machine learning models.

## 5.2 Dataset

Title: Heart Disease UCI Dataset

- Source: [UCI Machine Learning Repository](#)
- CSV containing 14+ features (age, sex, blood pressure, cholesterol, etc.) and a binary target (presence/absence of heart disease)

In this project we use the UCI Heart Disease dataset consisting of patient clinical attributes such as age, cholesterol, blood pressure, chest pain type, maximum heart rate, and other diagnostic features.

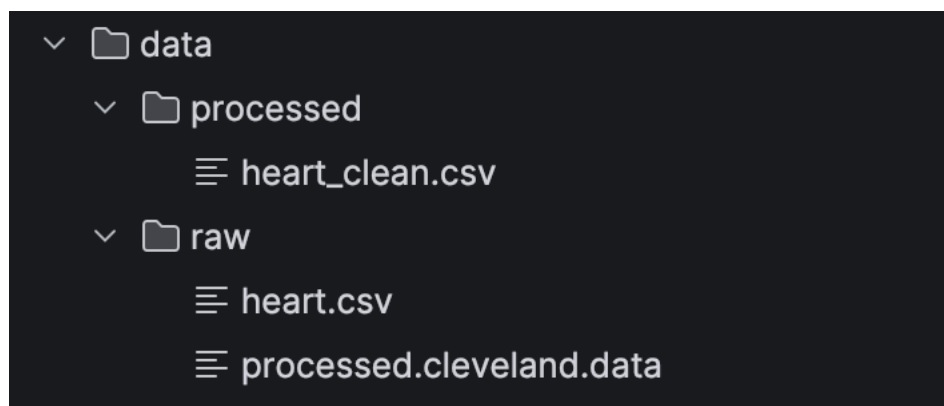
The target variable indicates the presence or absence of heart disease.

If predict=0 → No Heart Disease

If predict=1 → Heart Disease Detected

### Scripts/Files Used

- `scripts/download_dataset.py` – Downloads the Heart Disease dataset.
- `notebooks/01_EDA.ipynb` – Performs dataset loading, exploratory data analysis, and visualization



```
import pandas as pd
```

```
df = pd.read_csv("../data/raw/heart.csv")
```

```
df.head()
```

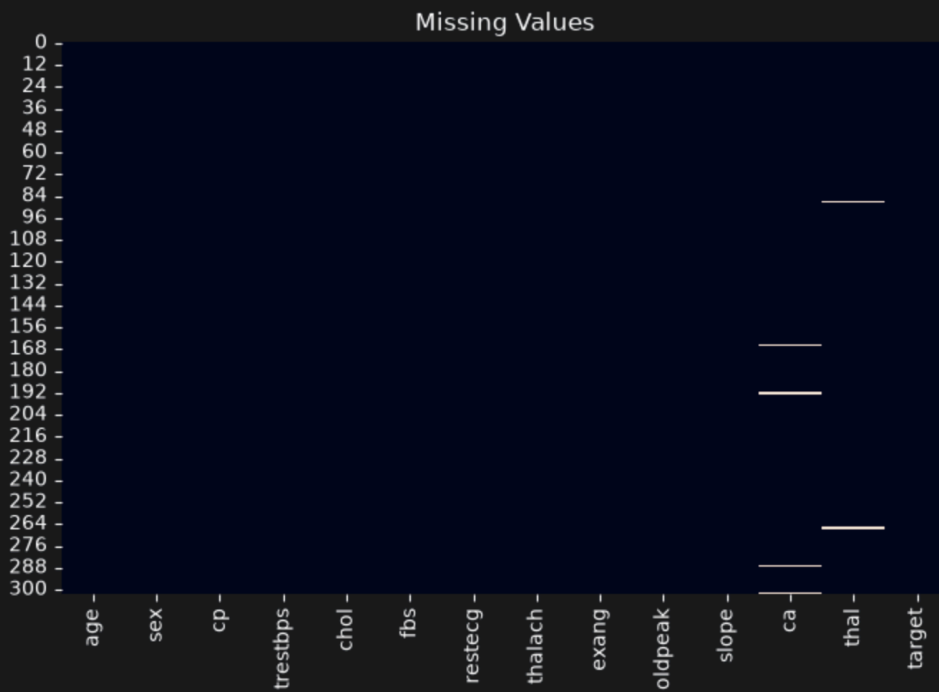
```
[1]
```

|   | age  | sex | cp  | trestbps | chol  | fbs | restecg | thalach | exang | oldpeak | slope | ca  | thal | target |
|---|------|-----|-----|----------|-------|-----|---------|---------|-------|---------|-------|-----|------|--------|
| 0 | 63.0 | 1.0 | 1.0 | 145.0    | 233.0 | 1.0 | 2.0     | 150.0   | 0.0   | 2.3     | 3.0   | 0.0 | 6.0  | 0      |
| 1 | 67.0 | 1.0 | 4.0 | 160.0    | 286.0 | 0.0 | 2.0     | 108.0   | 1.0   | 1.5     | 2.0   | 3.0 | 3.0  | 2      |
| 2 | 67.0 | 1.0 | 4.0 | 120.0    | 229.0 | 0.0 | 2.0     | 129.0   | 1.0   | 2.6     | 2.0   | 2.0 | 7.0  | 1      |
| 3 | 37.0 | 1.0 | 3.0 | 130.0    | 250.0 | 0.0 | 0.0     | 187.0   | 0.0   | 3.5     | 3.0   | 0.0 | 3.0  | 0      |
| 4 | 41.0 | 0.0 | 2.0 | 130.0    | 204.0 | 0.0 | 2.0     | 172.0   | 0.0   | 1.4     | 1.0   | 0.0 | 3.0  | 0      |

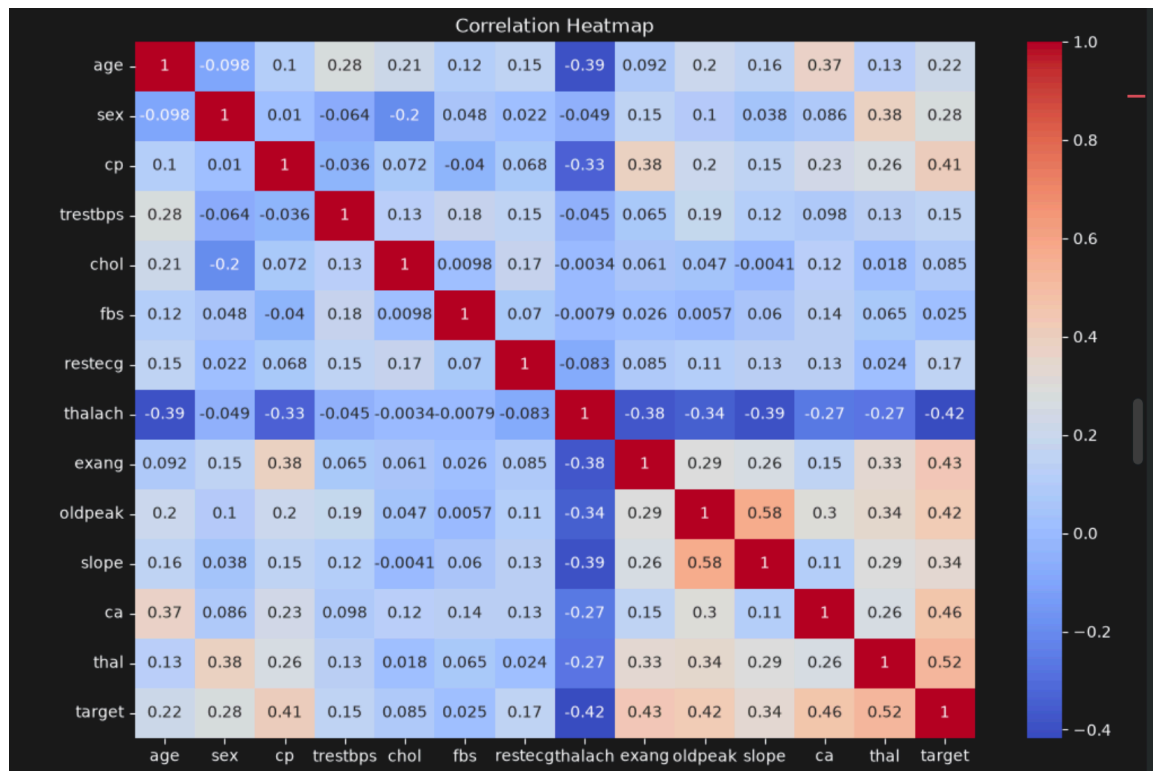
```
plt.title("Missing Values")
```

```
plt.show()
```

```
[4]
```



## Correlation heatmap:



## Cleaned Data Saved:

```
1 import os
2
3 # Create the processed directory if it doesn't exist
4 os.makedirs("../data/processed", exist_ok=True)
5
6 # Save cleaned dataset
7 df.to_csv("../data/processed/heart_clean.csv", index=False)
8
9 print("Cleaned dataset saved successfully!")
[12]
```

Cleaned dataset saved successfully!

## 6. Feature Engineering & Model Development

Prepare the dataset for model training and evaluate multiple machine learning algorithms

Implemented Logistic Regression and Random Forest models.  
Hyperparameter tuning was performed using GridSearchCV.

### Scripts/Files Used

- src/preprocess.py
- src/train.py
- src/evaluate.py
- notebooks/02\_Model\_Training\_v6\_using\_training\_evaluate.ipynb

### Model Performance:

Logistic Regression: Accuracy 88.52%, ROC-AUC 0.966

Random Forest: Accuracy 86.89%, ROC-AUC 0.943

Although Logistic Regression achieved a slightly higher test accuracy, the hyperparameter-tuned Random Forest model was selected as the final deployed model based on the model selection strategy implemented in the training pipeline.

## 7. MLflow Experiment Tracking

MLflow was used to record parameters, evaluation metrics, and trained models.

### Scripts/Files Used

- src/train.py

### Tool Used

- MLflow

```
(.venv) → mvanitha:~$ python3 -m src.train
Model saved to: /Users/mvanitha/Documents/heart-disease-prediction-mlops/models/best_model.pkl
Model logged to MLflow successfully!

***** Training Completed *****

{'Logistic Regression': {'Accuracy': 0.8852459016393442, 'Precision': 0.8387096774193549, 'Recall': 0.9285714285714286, 'F1-Score': 0.8813559322033898, 'ROC-AUC': 0.9664502164502166}, 'Random Forest': {'Accuracy': 0.8688524590163934, 'Precision': 0.8333333333333334, 'Recall': 0.8928571428571429, 'F1-Score': 0.8620689655172413, 'ROC-AUC': 0.9426406926406927}, 'Best Model': 'Random Forest', 'Best Parameters': {'classifier__max_depth': 5, 'classifier__min_samples_split': 5, 'classifier__n_estimators': 100}}
```

## Mflow screenshot:

Experiments > Heart Disease Prediction > Training runs

Q metrics.rmse < 1 and params.model = "tree" All time State: Active Datasets Sort: Created Columns + New run

Group by

|  | Run Name            | Created        | Dataset | Duration | Source   | Models     | accuracy     | f1_score     | precision    | recall       |
|--|---------------------|----------------|---------|----------|----------|------------|--------------|--------------|--------------|--------------|
|  | learned-ram-48      | 1 minute ago   | -       | 13.3s    | train.py | best_model | 0.8688524... | 0.8620689... | 0.8333333... | 0.8928571... |
|  | tasteful-snail-545  | 15 minutes ago | -       | 13.2s    | train.py | best_model | 0.8688524... | 0.8620689... | 0.8333333... | 0.8928571... |
|  | learned-yak-56      | 20 minutes ago | -       | 13.8s    | train.py | best_model | 0.8688524... | 0.8620689... | 0.8333333... | 0.8928571... |
|  | stylish-hound-608   | 26 minutes ago | -       | 13.0s    | train.py | best_model | 0.8688524... | 0.8620689... | 0.8333333... | 0.8928571... |
|  | masked-wasp-444     | 29 minutes ago | -       | 14.0s    | train.py | best_model | 0.8688524... | 0.8620689... | 0.8333333... | 0.8928571... |
|  | sedate-worm-886     | 38 minutes ago | -       | 16.3s    | train.py | best_model | 0.8688524... | 0.8620689... | 0.8333333... | 0.8928571... |
|  | tasteful-grouse-541 | 1 day ago      | -       | 13.2s    | train.py | best_model | 0.8688524... | 0.8620689... | 0.8333333... | 0.8928571... |

Heart Disease Prediction > Runs > sincere-croc-519

Overview Model metrics System metrics Traces Artifacts

Metrics (5)

| Metric    | Value              | Models     |
|-----------|--------------------|------------|
| accuracy  | 0.8688524590163934 | best_model |
| precision | 0.8333333333333334 | best_model |
| recall    | 0.8928571428571429 | best_model |
| f1_score  | 0.8620689655172413 | best_model |
| roc_auc   | 0.9426406926406927 | best_model |

Parameters (3)

| Parameter                     | Value |
|-------------------------------|-------|
| classifier__max_depth         | 5     |
| classifier__min_samples_split | 5     |
| classifier__n_estimators      | 100   |

Logged models (1)

| Model attributes |      |            |        |                |                   | No dataset         |
|------------------|------|------------|--------|----------------|-------------------|--------------------|
| Type             | Step | Model name | Status | Created        | Registered models | accuracy           |
| Output           | 0    | best_model | Ready  | 16 seconds ago | -                 | 0.8688524590163934 |

Created by mvanitha

Experiment ID 1

Status Finished

Run ID eeac951b65f441db8fecc3310e97497

Duration 13.7s

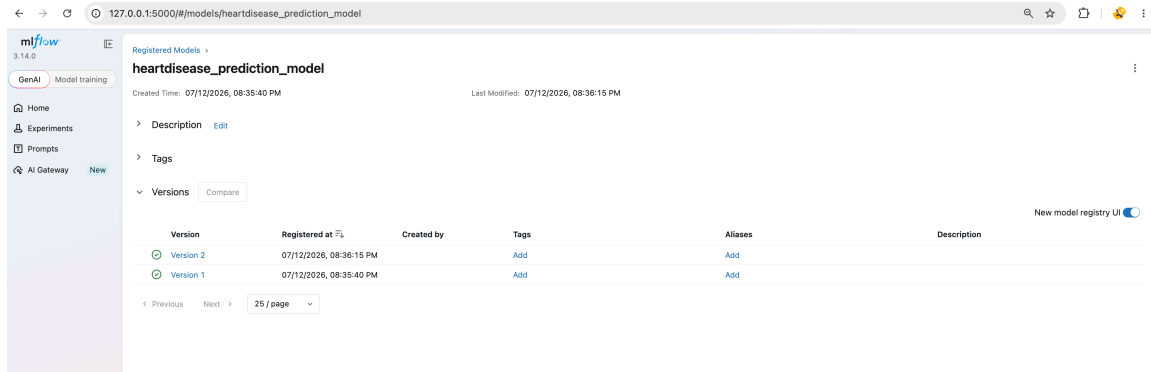
Source train.py

Registered prompts -

Datasets None

Tags Add tags

Registered models None

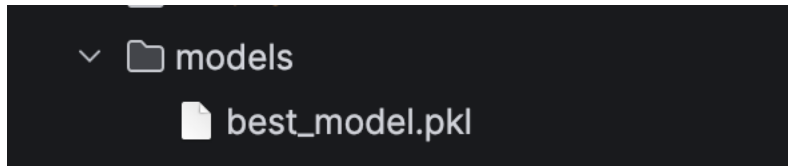


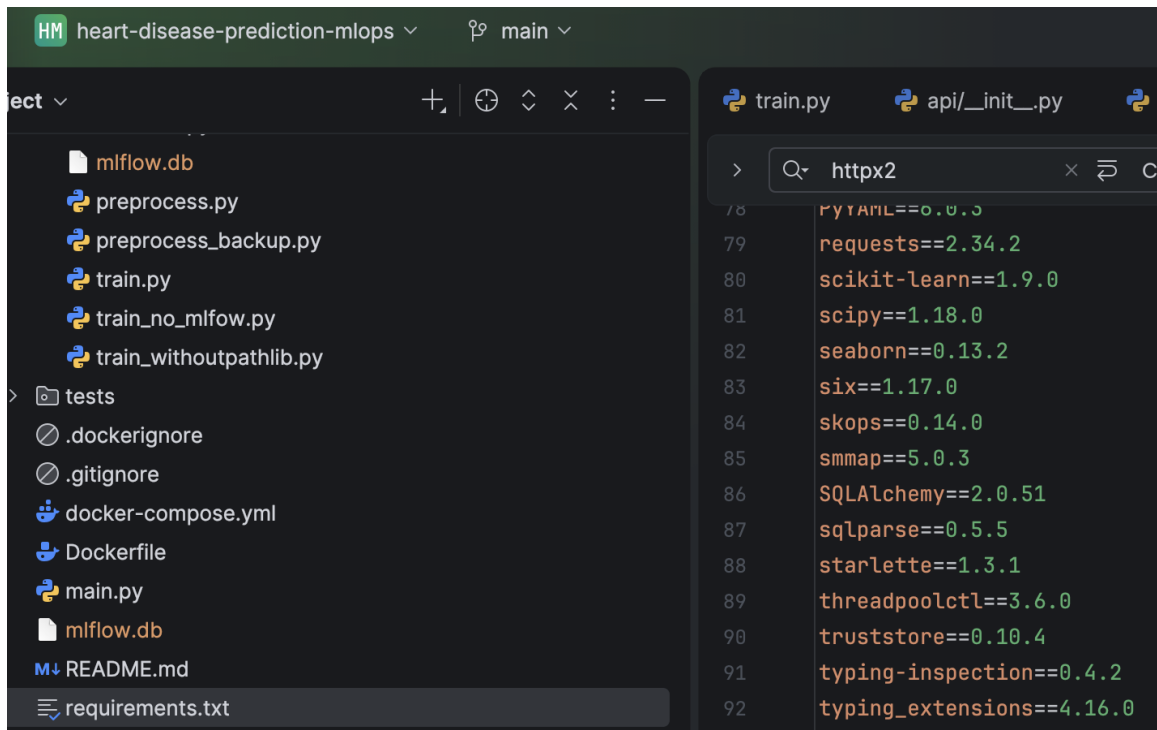
## 8. Model Packaging & Reproducibility

Package the trained model and maintain a reproducible environment.

### Files Used

- models/best\_model.pkl
- requirements.txt





## 9. FastAPI REST API

The trained model was exposed through a FastAPI application. Endpoints include '/', '/predict', and '/metrics'.

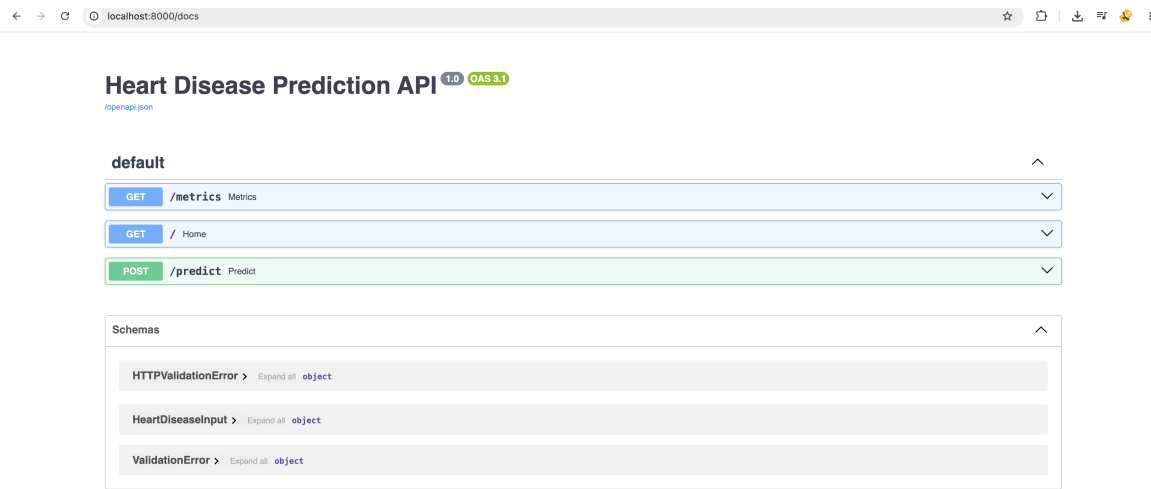
### Scripts/Files Used

- `api/app.py`
- `api/schemas.py`

### commands:

`uvicorn api.app:app`

`uvicorn api.app:app --reload`



## Body:

```
{
  "age": 72,
  "sex": 1,
  "cp": 0,
  "trestbps": 180,
  "chol": 420,
  "fbs": 1,
  "restecg": 2,
  "thalach": 80,
  "exang": 1,
  "oldpeak": 5.8,
  "slope": 2,
  "ca": 4,
  "thal": 3
}
```

## 10. Docker

The application was containerized using Docker to ensure portability across environments.

We Containerize the FastAPI application using Docker

## Files Used

- Dockerfile
- docker-compose.yml
- requirements.txt

## Commands Executed

`docker build -t heart-disease-api:v1.3 .`

`docker run -p 8000:8000 heart-disease-api:v1.3`

```
(.venv) → mvanitha:~$ docker images
```

| IMAGE                                                                                               | ID           | DISK USAGE | CONTENT SIZE | EXTRA |
|-----------------------------------------------------------------------------------------------------|--------------|------------|--------------|-------|
| gcr.io/k8s-minikube/kicbase:v0.0.50                                                                 | ffefc6978109 | 1.88GB     | 507MB        |       |
| gcr.io/k8s-minikube/kicbase@sha256:eb4fec00e8ad70adf8e6436f195cc429825ffb85f95afcdb5d8d9deb576f3e93 | eb4fec00e8ad | 1.88GB     | 507MB        | U     |
| heart-disease-api:latest                                                                            | 9cbb53298c75 | 1.32GB     | 283MB        |       |
| heart-disease-api:v1.3                                                                              | 9cbb53298c75 | 1.32GB     | 283MB        |       |
| mlopsmahe-heart-api:latest                                                                          | 3522f47725de | 1.3GB      | 281MB        | U     |

→ 127.0.0.1:8000/docs

### Heart Disease Prediction API 0.0 OAS 3.1

[OpenAPI JSON](#)

#### default

|      |          |         |
|------|----------|---------|
| GET  | /metrics | Metrics |
| GET  | /        | Home    |
| POST | /predict | Predict |

#### Schemas

HTTPValidationError > Expand all object

HeartDiseaseInput > Expand all object

ValidationError > Expand all object

## 11. Kubernetes Deployment

The Docker image was deployed on **Minikube** using Kubernetes Deployment and Service manifests. Port forwarding was used for local access.

## Files Used

- k8s/namespace.yaml
- k8s/deployment.yaml
- k8s/service.yaml

## Commands

```
kubectl apply -f k8s/
```

```
kubectl get pods -n heart-disease
```

```
kubectl get svc -n heart-disease
```

```
kubectl port-forward svc/heart-disease-service 8000:8000 -n  
heart-disease
```

```
(.venv) → mvanitha::~~$ kubectl get nodes
```

| NAME     | STATUS | ROLES         | AGE | VERSION |
|----------|--------|---------------|-----|---------|
| minikube | Ready  | control-plane | 18h | v1.35.1 |

```
(.venv) → mvanitha::~~$ kubectl get pods -n heart-disease
```

| NAME                              | READY | STATUS  | RESTARTS | AGE |
|-----------------------------------|-------|---------|----------|-----|
| grafana-646d894656-dh5c6          | 1/1   | Running | 0        | 46m |
| heart-disease-api-6dd84db7b-qvwz2 | 1/1   | Running | 0        | 59m |
| prometheus-794b4c48f9-q278f       | 1/1   | Running | 0        | 52m |

Curl

```
curl -X 'POST' \
  'http://localhost:8080/predict' \
  -H 'accept: application/json' \
  -H 'Content-Type: application/json' \
  -d '{
    "age": 72,
    "sex": 1,
    "cp": 0,
    "trestbps": 180,
    "chol": 420,
    "fbs": 1,
    "restecg": 2,
    "thalach": 80,
    "exang": 1,
    "oldpeak": 5.8,
    "slope": 2,
    "ca": 4,
    "thal": 3
  }'
```

Request URL

<http://localhost:8080/predict>

Server response

| Code | Details                                                                                                                                                                                                                                                          |
|------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 200  | <p>Response body</p> <pre>{   "prediction": 1,   "diagnosis": "Heart Disease Detected",   "probability": 0.6892 }</pre> <p>Response headers</p> <pre>content-length: 74 content-type: application/json date: Sat, 11 Jul 2026 21:54:16 GMT server: uvicorn</pre> |

Responses

| Code | Description         | Links    |
|------|---------------------|----------|
| 200  | Successful Response | No links |

Media type

Controls Accept header.

Example Value Schema

"string"

## 12. Monitoring & Logging

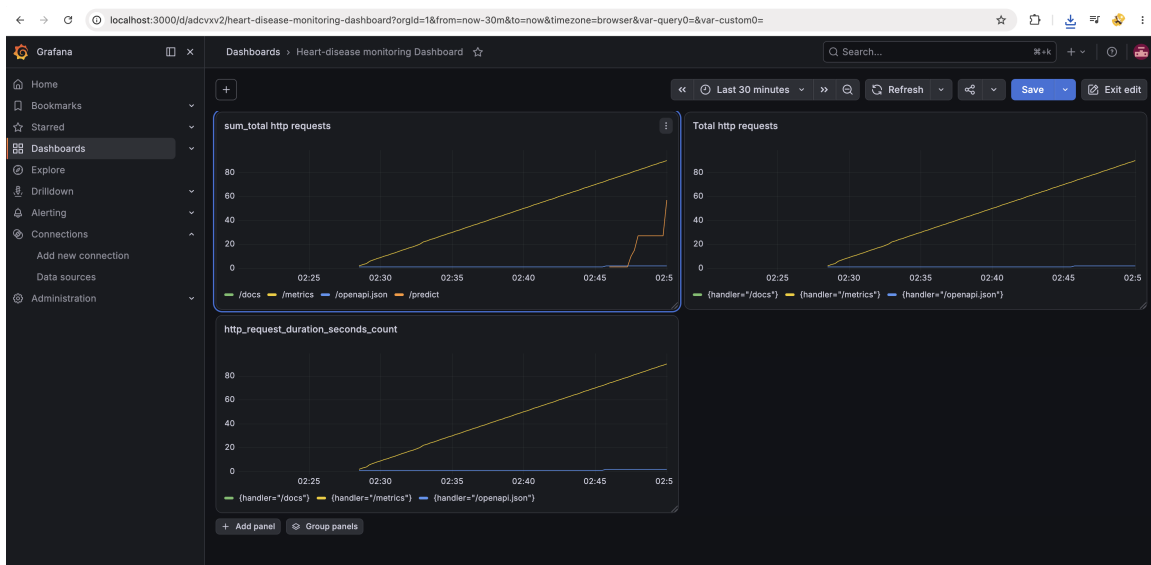
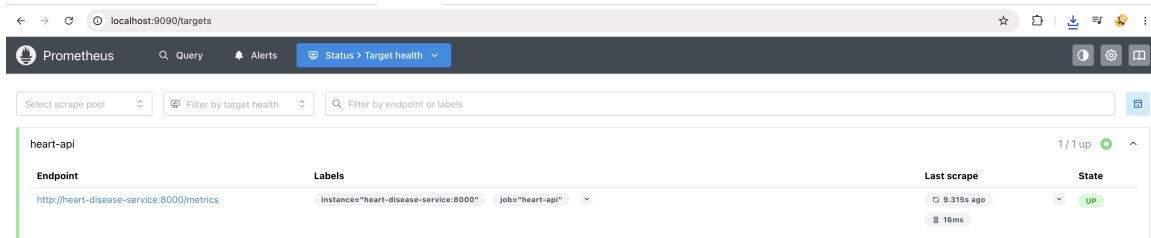
Prediction requests were logged using Python's logging module. Each request records the input, prediction, diagnosis, and probability.

Prometheus collected metrics from the FastAPI '/metrics' endpoint, while Grafana displayed request metrics through dashboards.

### Files Used

- api/app.py
- monitoring/prometheus-configmap.yaml

- monitoring/prometheus-deployment.yaml
- monitoring/prometheus-service.yaml
- monitoring/grafana-deployment.yaml
- monitoring/grafana-service.yaml



## 14. CI/CD Pipeline & Automated Testing

Unit tests were implemented using Pytest. GitHub Actions automatically runs the test suite whenever code is pushed.

### Files Used

- .github/workflows/ci.yml
- tests/test\_api.py

The screenshot shows the GitHub Actions interface for the repository '2024ad05057 / heart-disease-prediction-mlops'. The 'Actions' tab is active, displaying a list of workflow runs. The first run, 'clear the outputs in EDA and training python file', is highlighted. The interface includes a sidebar with navigation options like 'Code', 'Issues', 'Pull requests', 'Actions', 'Projects', 'Wiki', 'Security and quality', 'Insights', and 'Settings'. The main area shows a table of workflow runs with columns for Workflow, Event, Status, Branch, and Actor.

| Workflow                                                        | Event                                                      | Status  | Branch | Actor          |
|-----------------------------------------------------------------|------------------------------------------------------------|---------|--------|----------------|
| clear the outputs in EDA and training python file               | MLOps CI Pipeline #7: Commit 8d483df pushed by 2024ad05057 | Success | main   | 41 minutes ago |
| updated the read me                                             | MLOps CI Pipeline #6: Commit 75d4778 pushed by 2024ad05057 | Success | main   | Today at 17:55 |
| github-actions image added                                      | MLOps CI Pipeline #5: Commit 8c05245 pushed by 2024ad05057 | Success | main   | Today at 16:59 |
| updated the predict image in read.me                            | MLOps CI Pipeline #4: Commit 5d7383d pushed by 2024ad05057 | Success | main   | Today at 16:49 |
| ignore MLflow database                                          | MLOps CI Pipeline #3: Commit 8d03d77 pushed by 2024ad05057 | Success | main   | Today at 15:39 |
| Remove IDE configuration files                                  | MLOps CI Pipeline #2: Commit 72b7c88 pushed by 2024ad05057 | Success | main   | Today at 15:38 |
| First commit mahe: End-to-end Heart Disease MLOps pipeline code | MLOps CI Pipeline #1: Commit d72a2b pushed by 2024ad05057  | Success | main   | Today at 15:24 |

The screenshot shows the details of the workflow run 'clear the outputs in EDA and training python file #7'. The 'Run Unit Tests' step is expanded, displaying the test results and logs. The logs show the execution of 'Run pytest -v' and the successful completion of the tests. The interface includes a sidebar with navigation options like 'Summary', 'Annotations', 'All jobs', 'test', 'Run details', 'Usage', and 'Workflow file'. The main area shows the test results and logs for the 'Run Unit Tests' step.

```

1  Run pytest -v
11 ===== test session starts =====
12 platform linux -- Python 3.13.14, pytest-9.1.1, pluggy-1.6.0 -- /opt/hostedtoolcache/Python/3.13.14/x64/bin/python
13 cachedir: .pytest_cache
14 rootdir: /home/runner/work/heart-disease-prediction-mlops/heart-disease-prediction-mlops
15 plugins: anyio-4.14.1
16 collecting ... collected 3 items
17
18 tests/test_api.py::test_home PASSED [ 33%]
19 tests/test_api.py::test_prediction PASSED [ 66%]
20 tests/test_preprocess.py::test_create_preprocessor PASSED [100%]
21
22 ===== warnings summary =====
23 .../opt/hostedtoolcache/Python/3.13.14/x64/lib/python3.13/site-packages/joblib/numpy_pickle.py:207: 412 warnings
24 /opt/hostedtoolcache/Python/3.13.14/x64/lib/python3.13/site-packages/joblib/numpy_pickle.py:207: DeprecationWarning: Setting the shape on a Numpy array has been deprecated in Numpy 2.5.
25 As an alternative, you can create a new view using np.reshape (with copy=False if needed).
26     array.shape = self.shape
27
28 -- Docs: https://docs.pytest.org/en/stable/how-to/capture-warnings.html
29 ===== 3 passed, 412 warnings in 1.80s =====

```

## 15. Results

The deployed API successfully predicts heart disease, is containerized, orchestrated using Kubernetes, monitored through Prometheus and Grafana, and validated using automated tests.

Curl

```
curl -X 'POST' \
  'http://localhost:8000/predict' \
  -H 'accept: application/json' \
  -H 'Content-Type: application/json' \
  -d '{
    "age": 72,
    "sex": 1,
    "cp": 0,
    "trestbps": 180,
    "chol": 420,
    "fbs": 1,
    "restecg": 2,
    "thalach": 80,
    "exang": 1,
    "oldpeak": 5.8,
    "slope": 2,
    "ca": 4,
    "thal": 3
  }'
```

Request URL

<http://localhost:8000/predict>

Server response

Code Details

200

Response body

```
{
  "prediction": 1,
  "diagnosis": "Heart Disease Detected",
  "probability": 0.6892
}
```

Response headers

```
content-length: 74
content-type: application/json
date: Sat, 11 Jul 2026 21:54:16 GMT
server: uvicorn
```

Responses

| Code | Description         | Links    |
|------|---------------------|----------|
| 200  | Successful Response | No links |

Media type

Controls Accept header.

Example Value Schema

```
"string"
```

localhost:8000/docs#/default/predict\_predict\_post

Curl

```
curl -X 'POST' \
  'http://localhost:8000/predict' \
  -H 'accept: application/json' \
  -H 'Content-Type: application/json' \
  -d '{
    "age": 63,
    "sex": 1,
    "cp": 3,
    "trestbps": 145,
    "chol": 233,
    "fbs": 1,
    "restecg": 0,
    "thalach": 150,
    "exang": 0,
    "oldpeak": 2.3,
    "slope": 0,
    "ca": 0,
    "thal": 1
  }'
```

Request URL

<http://localhost:8000/predict>

Server response

Code Details

200

Response body

```
{
  "prediction": 0,
  "diagnosis": "No Heart Disease",
  "probability": 0.3704
}
```

Response headers

```
content-length: 68
content-type: application/json
date: Sat, 11 Jul 2026 21:46:25 GMT
server: uvicorn
```

Responses

| Code | Description         | Links    |
|------|---------------------|----------|
| 200  | Successful Response | No links |

Media type

## 16. Conclusion

This project demonstrates a complete end-to-end MLOps pipeline, covering model development, deployment, monitoring, logging, testing, and continuous integration. The integration of MLflow, FastAPI, Docker, Kubernetes, Prometheus, Grafana, and GitHub Actions provides a production-ready solution that follows modern MLOps practices

## References

UCI Machine Learning Repository – Heart Disease Dataset

<https://archive.ics.uci.edu/dataset/45/heart+disease>

<https://devopscube.com/devops-to-mlops/>

FastAPI Documentation: <https://devdocs.io/fastapi/>

MLflow Documentation: <https://mlflow.org/docs/latest/>

Docker Documentation: <https://docs.docker.com/>

Kubernetes Documentation

Prometheus and Grafana Documentation

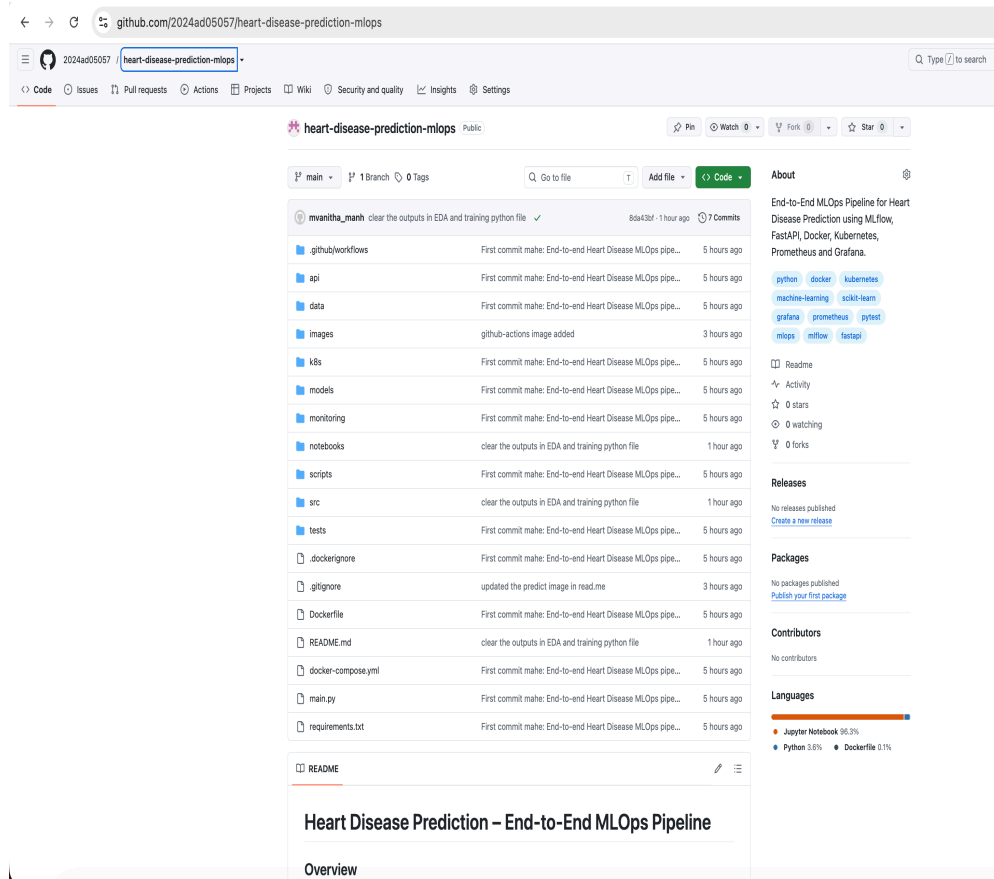
GitHub Actions Documentation

## Project structure:

```
(.venv) + mvanitha::~$ tree -L 2 -I '.git|.venv|__pycache__|mlruns|*.pyc'
.
├── Dockerfile
├── README.md
├── api
│   ├── __init__.py
│   ├── app.py
│   └── schemas.py
├── data
│   ├── processed
│   └── raw
├── docker-compose.yml
├── images
│   ├── github-actions.png
│   ├── grafana.png
│   ├── kubernetes.png
│   ├── mlflow.png
│   ├── predict.png
│   ├── predict_1.png
│   ├── prometheus.png
│   ├── swagger.png
│   └── vscode.png
├── k8s
│   ├── deployment.yaml
│   ├── namespace.yaml
│   └── service.yaml
├── main.py
├── mlflow.db
├── models
│   └── best_model.pkl
├── monitoring
│   ├── grafana-deployment.yaml
│   ├── grafana-service.yaml
│   ├── prometheus-config.yaml
│   ├── prometheus-deployment.yaml
│   ├── prometheus-service.yaml
│   └── prometheus.yml
├── notebooks
│   ├── 01_EDA.ipynb
│   ├── 02_Model_Training.ipynb
│   ├── 02_Model_Training_v2.ipynb
│   ├── 02_Model_Training_v3.ipynb
│   ├── 02_Model_Training_v4.ipynb
│   ├── 02_Model_Training_v5.ipynb
│   └── 02_Model_Training_v6_using_training_evaluate.ipynb
├── requirements.txt
├── scripts
│   └── download_dataset.py
├── src
│   ├── __init__.py
│   └── evaluate.py
```

```
02_Model_Training_v6_using_training_evaluate.ipynb
├── requirements.txt
├── scripts
│   └── download_dataset.py
├── src
│   ├── __init__.py
│   ├── evaluate.py
│   ├── mlflow.db
│   ├── preprocess.py
│   ├── preprocess_backup.py
│   ├── train.py
│   ├── train_no_mlflow.py
│   └── train_withoutpathlib.py
├── tests
│   ├── __init__.py
│   ├── test_api.py
│   └── test_preprocess.py
```

## Github folder structure screensho



t:

## Commands Summary

source .venv/bin/activate

python src/train.py

mlflow ui

uvicorn api.app:app --reload

docker images

```
docker ps
```

```
kubectl get ns
```

```
kubectl get pods -n heart-disease
```

```
kubectl get svc -n heart-disease
```

```
kubectl port-forward svc/prometheus 9090:9090 -n heart-disease
```

```
kubectl port-forward svc/grafana 3000:3000 -n heart-disease
```